

Tab-HGNN: Learning Column Representation with Heterogeneous Graph Neural Network for Web Table Interpretation

Xingyu Su
Shanghai Jiao Tong University
Shanghai, China
xingyusu@sjtu.edu.cn

Yanyan Shen*
Shanghai Jiao Tong University
Shanghai, China
sheny@sjtu.edu.cn

Abstract—Web tables are valuable resources that can enrich existing knowledge bases. Researchers try to make the predictions of columns and column pairs in web tables into entities and relations of knowledge bases, which are known as the Column Type Annotation (CT) and Relation Extraction (RE) tasks in web table interpretation. Learning useful and accurate column vector representations plays the central role in solving the two tasks. A column’s semantics can be determined by three kinds of relationships: Column-Cell, Column-Table, Column-Column relationships. Existing works only rely on the Column-Cell relationship to determine the semantics of columns and result in suboptimal performances. In this paper, we propose to solve the above CT and RE tasks with the heterogeneous graph neural network technique. First, we construct heterogeneous graphs for web tables. Next, a new model named Tab-HGNN is proposed to consider all the three kinds of relationships and learn target column representations. Extensive experiments on real-world web table datasets demonstrate the effectiveness of the proposed Tab-HGNN model on the CT and RE tasks. It outperforms the competitive baselines by achieving up to +7.9% and +1.6% macro F_1 scores, +2.6% and +0.2% weighted F_1 scores on two CT datasets, respectively. It also achieves +0.3% macro F_1 scores, +0.1% weighted F_1 scores on one RE dataset.

Index Terms—Table interpretation, graph neural network, heterogeneous graph

I. INTRODUCTION

Recent years have witnessed the emergence of large knowledge bases (e.g., Freebase [1], DBPedia [2]) that are utilized in various downstream tasks such as question answering, faceted search and data mining. To enrich knowledge bases, a number of advanced Natural Language Processing (NLP) tools [3]–[5] have been developed to extract entities and relations from massive text data on web pages. In addition to unstructured text information, web tables are the most common and powerful source to augment knowledge bases [6], [7]. A web table is generally structured into rows and columns and involves heterogeneous information such as table caption, column header and cell mention. The alignment between a web table and a knowledge base requires to determine the entity type of each column and the relation between columns such that triplets can be extracted from the web table to

* corresponding author.

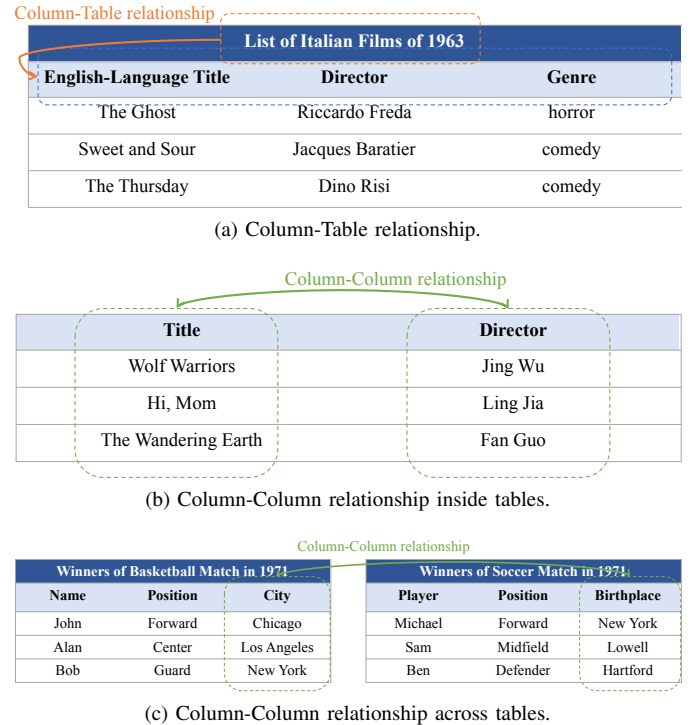


Fig. 1. Motivating examples showing the importance of Column-Table relationship and Column-Column relationship.

populate the knowledge base. This process is also known as *web table interpretation* [8]. In this paper, we aim to tackle two important tasks in web table interpretation, namely Column Type Annotation (CT) and Relation Extraction (RE), which determine the types of columns and column pairs, respectively.

Learning useful and accurate column vector representations plays the central role in solving the CT and RE tasks for web table interpretation [8], [9]. Existing works proposed to first learn cell representations by CNN [11], attention network [8] or transformers [9], [22], and then obtained a column’s representation by performing “pooling operation” (e.g., average pooling [8], [9]) over the representations of its containing cells. Such methods actually utilize the Column-Cell relationship in

web tables and assume a column’s semantics is determined by its containing cells. While this kind of relationship is important, we argue that there are other kinds of relationships that are useful to disclose column semantics towards better representations.

Motivating Example 1. Suppose our task is to annotate the column types for the table in Fig. 1a. Take the first column as an example. Given the “The Ghost”, “Sweet and Sour”, “The Thursday” and the header “English-Language Title”, it is hard to conclude that the first column talks about films. The table title “List of Italian Films of 1963” can help us infer the column type “Film Titles” of the first column. We call this kind of inference from table titles to column types the Column-Table relationship.

Motivating Example 2. Fig. 1b shows one table containing two columns “Title” and “Director”. Suppose we want to predict the column type “Film Titles” of the first column. However, only knowing the cells “Wolf Warriors”, “Hi, Mom”, “The Wandering Earth” and the header “Title” is insufficient to make a correct prediction. If we know the second column talks about “Film Director”, such information can narrow down the semantics of the first column to film-related objects and is useful to predict the column type as “Film Titles”. Since the above process happens within the table, we call such an inference from one column to other column(s) the Column-Column relationship within a table.

Motivating Example 3. Fig. 1c shows two tables each having one column about cities. It is difficult to predict the type “City” for the third column in the second table, which involves the column header “Birthplace” and three cells “New York”, “Lowell”, “Hartford”. However, we find it is easier for the first column in the first table to be classified as “City”, since its header is “City” and the cells are all famous cities. Furthermore, these two columns should have similar column representations because they have overlapping cells, similar table titles, surrounding cells, etc. Building Column-Column relationship between two similar columns is beneficial for the second column to be classified as “City”. We call this kind of relationship the Column-Column relationship across tables.

In this paper, we aim to consider all the three different kinds of relationships, namely Column-Cell, Column-Column and Column-Table relationships, to solve the CT and RE tasks. There are two challenging issues to be tackled. The first challenge is: how to model different relationships when learning column representations? Different kinds of relationships have different semantics, e.g., the Column-Table relationship has the “belonging” semantics while the Column-Column relationship has the “similarity” semantics. It is desirable to retain the heterogeneity among different kinds of relationships and allow them to augment each other for solving the CT and RE tasks. The second challenge is: how to accurately identify semantically related columns across tables to enforce the Column-Column relationship and incorporate the relationship into the learning process? Since we want to align the representations of the related columns, referring to columns that are not closely related with the target column will degenerate the quality of

the column representation.

To solve the above challenges, we first convert web tables into a heterogeneous graph to model the three kinds of heterogeneous relationships in a unified way. After that, we propose a new heterogeneous graph neural network model named Tab-HGNN consisting of the Two-Stage Message Passing Module, the Edge Generation Module and the Prediction Module. Specifically, the First-Stage Message Passing Module models the Column-Cell relationship and Column-Table relationship and gets primary column representations containing the information within a table. After the first-stage, the Edge Generation Module tries to build edge connections between semantically related columns. The edge connections have learnable parameters and are dynamically updated based on the similarity measurement over primary column representations. Next, the Second-Stage Message Passing Module updates the primary column representations by edges between columns to model the Column-Column relationship and gets the final column representations. Finally, the Prediction Module predicts the types based on the final column representations. We conduct experiments on real-world web tables. The experimental results show that our proposed Tab-HGNN outperforms the state-of-the-art methods in both Column Type Annotation and Relation Extraction tasks.

To summarize, the main contributions of this paper are the following:

- We develop a new heterogeneous graph neural network model named Tab-HGNN to fully utilize three kinds of relationships including Column-Cell, Column-Table and Column-Column relationships, to solve the Column Type Annotation and Relation Extraction tasks in web table interpretation.
- We propose to dynamically construct connections between columns to model the Column-Column relationship during the training process, which improves the web table interpretation performance.
- We conduct extensive experiments on real-world web table datasets. The results show that our proposed Tab-HGNN outperforms the state-of-the-art methods by achieving +7.9% and +1.6% macro F_1 scores, +2.6% and +0.2% weighted F_1 scores on two CT datasets, respectively. It also achieves +0.3% macro F_1 scores, +0.1% weighted F_1 scores on one RE dataset.

The rest of this paper is organized as follows. Section II discusses the related works for web table interpretation. Section III presents the preliminaries. Section IV introduces the heterogeneous graph construction and our proposed Tab-HGNN model in detail. Section V shows the experimental results. We conclude this paper in Section VI.

II. RELATED WORK

Various researches [7], [29], [30] have proposed to leverage web tables to enrich the knowledge bases by solving the CT and RE tasks. Early works [10], [12] tried to design hand-crafted engineered features to form column representations.

Sherlock [10] extracted 1588 features based on global statistics, character distribution, word embeddings and paragraph vectors for each column, and used feed-forward layers of deep learning to generate the column representations. Sato [12] suggested to use Latent Dirichlet Allocation (LDA) topic model to consider the table topic information on top of the column representations of Sherlock. However, both methods rely on shallow hand-crafted features and cannot have a good generalization to other datasets, which would yield suboptimal the prediction performance.

Recently, many works [8], [11] have leveraged the deep learning methods to get column representations. They tried to first learn good cell representations and then aggregated cell representations to get column representations. HNN [11] used RNN model to encode the word sequence of each cell mention and applied CNN model to consider row and column neighbors to learn good cell representations. It could further incorporate external knowledge bases to improve the prediction performance. TCN [8] suggested to consider both intra-table and inter-table information. It leveraged inter-table cells of the same value, same schema position and same page topic to enhance cell representations. While the Column-Cell relationship is important to form the final column representations, those methods ignore the Column-Table relationship and Column-Column relationship that are useful to regulate the column semantics towards better representations.

Another line of research used transformer-based model to learn column representations [9], [17], [20], [22]. TaBERT [22] encoded selected rows of tabular data by BERT-like model. TURL [9] suggested visibility matrix in transformers to consider the row-column structure for tabular data. They also proposed Masked Entity Recovery objective to learn cell mention representations. TABBIE [17] used two transformers on rows and columns to learn their embeddings. TUTA [20] used tree-like structure to encode general structured tables. However, those methods still focus on learning cell representations, without the direct modeling of the Column-Table and Column-Column relationships.

III. PRELIMINARIES

In this work, we focus on a collection $\mathcal{T}$ of web tables that can enrich a given knowledge base with an entity type set $\mathcal{C}$ and a relation type set $\mathcal{R}$. A web table T in $\mathcal{T}$ typically consists of three parts: (1) Table topic P , which often contains page title, section title and table caption. (2) Headers H , which include the names of columns. (3) Cell mentions E , which refer to the contents of cells. For a table T , we denote its columns as $c_1, \dots, c_{L_c^T}$ and the rows as $r_1, \dots, r_{L_r^T}$ where L_c^T and L_r^T is the column number and the row number of T , respectively.

We now formally define the CT and RE tasks as follows.

Definition 3.1 (Column Type Annotation): For any column c_i of a web table T , Column Type Annotation task is to assign the column c_i to its type $l_c \in \mathcal{C}$, where $\mathcal{C}$ is the set of entity types.

Definition 3.2 (Relation Extraction): For any column pair $\{c_i, c_j\}$ of a web table T , Relation Extraction task aims to assign the column pair $\{c_i, c_j\}$ to its type $l_r \in \mathcal{R}$, where $\mathcal{R}$ is the set of relation types.

IV. METHODOLOGY

Our goal is to improve the column representations by exploiting the Column-Cell, Column-Table and Column-Column relationships. We propose to construct a heterogeneous graph to capture three kinds of relationships in a unified manner and develop a heterogeneous graph neural network named Tab-HGNN to achieve the goal. The construction of the heterogeneous graph and the pipeline of Tab-HGNN are shown in Fig. 2.

- We convert the collection of web tables into a heterogeneous graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \tau, \phi\}$, where τ is the node type mapping function and ϕ is the edge type mapping function. $\mathcal{V}$ has three node types which are table, column, cell nodes and $\mathcal{E}$ has five different edge types.
- The First-stage Message Passing Module performs heterogeneous graph feature propagation following the Column-Cell and Column-Table relationships. It produces the primary column representations h_c^o .
- The Edge Generation Module establishes edges between column nodes based on the semantic similarity among the primary column node representations.
- The Second-stage Message Passing Module updates the primary column representations and produces the final column and column pair representations h_c^f and h_p^f following the Column-Column relationship.
- The Prediction Module will generate the predictions for the CT and RE tasks based on the primary and final column and column pair representations.

The pseudocode of the whole learning process is shown in Algorithm 1.

A. Heterogeneous Graph Construction

To capture the Column-Cell, Column-Column and Column-Table relationships, we generate a heterogeneous graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \tau, \phi\}$ for a collection of web tables $\mathcal{T}$ and initialize the embeddings of the nodes and edges. $\mathcal{V}$ is the node set consisting of three types of nodes: cell nodes $\mathcal{V}_e$, column nodes $\mathcal{V}_c$, and table nodes $\mathcal{V}_t$ (we merge the same cell mention in the same column to one node). We create edges between those nodes. The edges construction and the initialization of node and edge features will be presented in the following.

1) *Edge Construction:* Consider a collection of web tables $\mathcal{T}$ and one specific table $T \in \mathcal{T}$ with L_r rows and L_c columns. We have cell nodes $\mathcal{V}_e$, column nodes $\mathcal{V}_c$ and table nodes $\mathcal{V}_t$. We establish five kinds of edges among these nodes which can be seen in Fig. 2. Those edges include:

- $\mathcal{E}_{t2c}$ from table nodes to column nodes to model the Column-Table relationship. All the column nodes c^T in web table T are connected with the belonging table node. These edges are used to propagate the table information to the column nodes.

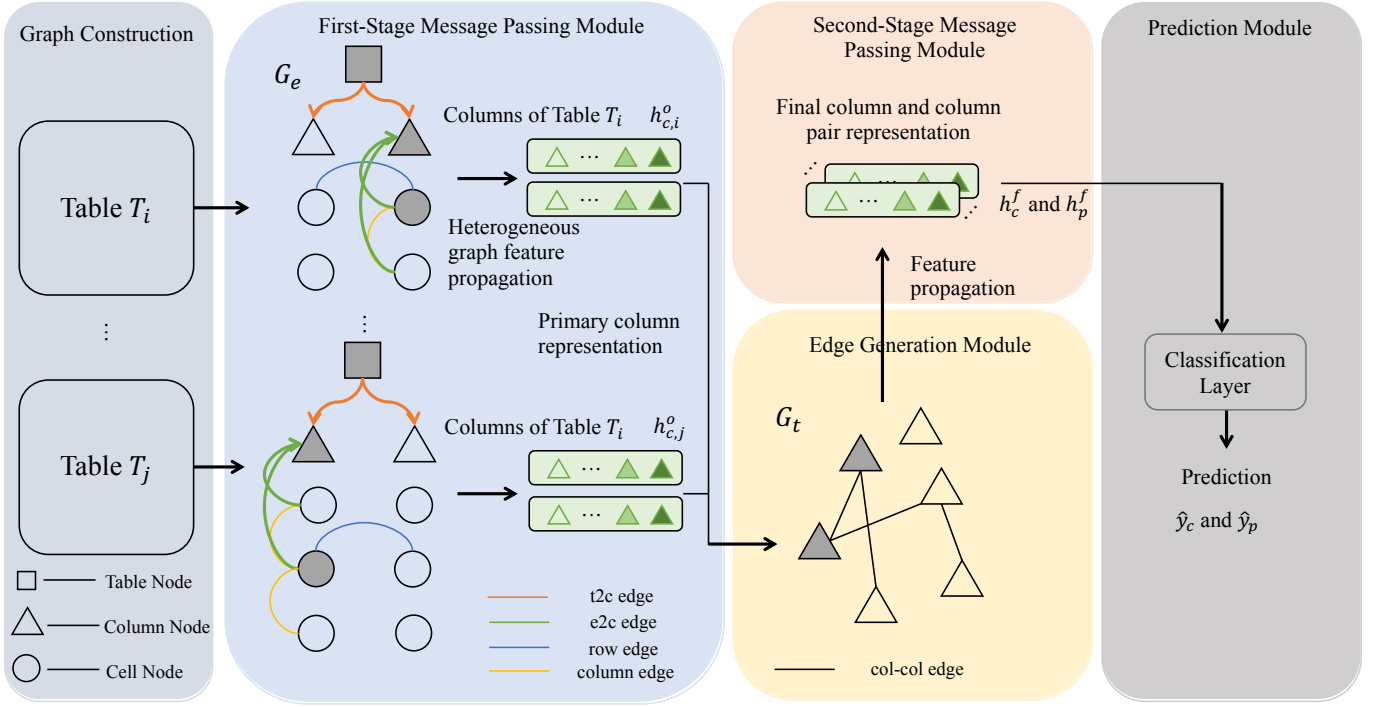


Fig. 2. The construction of a heterogeneous graph and the pipeline of the proposed model Tab-HGNN consisting of the Two-Stage Message Passing Module, the Edge Generation Module and the Prediction Module.

Algorithm 1 The Training Algorithm of Tab-HGNN

Input: heterogeneous graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \tau, \phi\}$

Output: learned model parameters θ

```

1: for each training iteration do
2:   for a batch of table nodes  $\mathcal{V}_t$  of  $\mathcal{G}$  do
3:     Generate the batch of column nodes  $\mathcal{V}_c$  of the batch
       of table nodes  $\mathcal{V}_t$ ;
4:     First-Stage feature propagation to get primary column
       representations  $h_c^o$  by Equation 3-7;
5:     Create edges between column nodes based on pri-
       mary column node embeddings by Equation 8-11;
6:     Second-Stage feature propagation to get final column
       representations  $h_c^f$  by Equation 12;
7:     Generate final column pair representations  $h_p^f$  by
       Equation 13;
8:     Generate predictions  $\hat{y}_c$  and  $\hat{y}_p$  by Equation 14;
9:     Calculate multi-task loss  $L$  by Equation 17;
10:    Perform back-propagation;
11:    Update parameters  $\theta$ ;
12:  end for
13: end for

```

- $\mathcal{E}_{e2c}$ from cell nodes to column nodes in order to model the Column-Cell relationship. For each column c_j^T in web table T , we connect the column node with all the containing cell nodes $e_{:,j}^T$. These edges are to extract the features of the cells to form the column representations.

- $\mathcal{E}_{row}$ between cell nodes of the same row. Consider a cell $e_{i,j}^T$ in the i -th row and j -th column of the web table T , and $e_{i,:}^T$ refers to other cells of the same row. We build connections between cell nodes for the same row since they belong to one specific record.
- $\mathcal{E}_{col}$ between cell nodes of the same column. The cells of the same column are $e_{:,j}^T$ for cell $e_{i,j}^T$. Cells of the same column belong to the same semantic type, so we build connections between cell nodes for the same column.
- $\mathcal{E}_{col-col}$ between related column nodes. $\mathcal{E}_{col-col}$ includes two parts. The first models the Column-Column relationship inside tables. For every two column nodes c_i^T and c_j^T of one specific table T , we build connections between them. The second models the Column-Column relationship across tables. We do not have any predefined edges and it is dynamically learned based on the column node representations during the learning process. The detailed explanation on the construction process is described in the Edge Generation Module.

2) *Embedding Initialization*: Here we show how to initialize the embedding of nodes and edges in the heterogeneous graph. The embeddings $\mathbf{h}_{e_{i,j}} \in \mathbb{R}^D$ of any cell node $e_{i,j}$ can be randomly initialized or by the Word2Vec or GloVe embeddings [14], [15]. If there are multiple words inside the cell $e_{i,j}$, we compute the average embedding as follows:

$$\mathbf{h}_{e_{i,j}} = \frac{1}{L_w} \sum_{k=1}^{L_w} \mathbf{h}_{w_k^{i,j}}, \quad (1)$$

where $\mathbf{h}_{w_{i,j}^k}$ is the embedding of k -th word w_k in the cell $e_{i,j}$, and L_w is the number of words. We use zero embedding for empty cells and randomly initialize the embeddings for unknown words.

For each column node embedding $\mathbf{h}_c \in \mathbb{R}^{D_i}$ and table node embedding $\mathbf{h}_t \in \mathbb{R}^{D_i}$, we use the word embeddings of the headers and titles, respectively. The titles are the concatenation of the page title, section title and table caption. We still take the average embedding of the words in a title. If a column does not have a header, we initialize the column embedding by the average embedding of the cell mentions in the column. Similarly, if there is no title, we initialize the title embedding by the average of column embeddings:

$$\mathbf{h}_c = \frac{1}{L_r} \sum_{i=1}^{L_r} \mathbf{h}_{e_{i,j}}, \quad \mathbf{h}_t = \frac{1}{L_c} \sum_{j=1}^{L_c} \mathbf{h}_{c_j}, \quad (2)$$

where $\mathbf{h}_{e_{i,j}}$ is the cell embedding of i -th row of the column j and $\mathbf{h}_{c_j}$ is the embedding of j -th column. L_r is the row number and L_c is the column number.

In order to incorporate edge type information of the heterogeneous graph, we choose to randomly initialize the embedding for each edge type, meaning the edges sharing the same edge type have the same initial embedding.

B. First Stage of Message Passing Module

As shown in Fig. 2, the message passing module has two stages. The first stage tries to learn the primary column representations $\mathbf{h}_c^o$. Here the heterogeneous graph feature propagation only uses the edges in $\mathcal{E}_{t2c}$, $\mathcal{E}_{e2c}$, $\mathcal{E}_{row}$ and $\mathcal{E}_{col}$. The feature propagation process will be illustrated in three steps: attention calculation, message producing and information aggregation.

1) *Attention Calculation*: Inspired by SimpleHGN [19] which is used for the modeling of heterogeneous graphs, we first calculate attention between node i and node j based on their features $\mathbf{h}_i$, $\mathbf{h}_j$, and the edge features $\mathbf{r}_{ij}$ between them:

$$att_{i,j} = \text{LeakyReLU}(a^\top [\mathbf{W}_\tau(i)\mathbf{h}_i \parallel \mathbf{W}_\tau(j)\mathbf{h}_j \parallel \mathbf{W}_r\mathbf{r}_{ij}]), \quad (3)$$

where $\mathbf{W}_\tau$ is the feature projection matrix for different node types τ and $\mathbf{W}_r$ is edge projection matrix. $a \in \mathbb{R}^{D_n} \times \mathbb{R}^{D_n} \times \mathbb{R}^{D_r}$ is a parametrized vector for calculating the similarity score between node i and j .

The softmax operation is applied on top of the Equation 3:

$$\alpha_{i,j} = \frac{\exp(att_{i,j})}{\sum_{k \in N_i} \exp(att_{i,k})}, \quad (4)$$

where $\alpha_{i,j}$ is the final attention score between node i and j .

2) *Message Producing*: While calculating the attention scores, the message is produced by source node j for target node i :

$$\mathbf{m}_{i,j} = \mathbf{W}_{e,\phi(ij)} \mathbf{W}_{m,\tau(j)} \mathbf{h}_j, \quad (5)$$

where $\mathbf{W}_{m,\tau(j)}$ is the message producing matrix for node j . Besides, the message produced by the source nodes can vary

for different edges. Thus, another weight matrix $\mathbf{W}_{e,\phi(ij)}$ is applied for the message producing.

3) *Information Aggregation*: We use the multi-head attention mechanism [18], [19] to update the representation of node i in layer $l+1$:

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\parallel \sum_{k=1}^K \alpha_{i,j,k}^{(l)} \mathbf{m}_{i,j,k}^{(l)} + \mathbf{W}_{\tau(i),res,k}^{(l)} \mathbf{h}_{i,k}^{(l)} \right), \quad (6)$$

where $\mathbf{W}_{\tau(i),res}^{(l)}$ is trainable matrix for the residual connection and K is the number of heads.

For the final layer, we average the representations:

$$\mathbf{h}_i^{(L)} = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in N_i} \alpha_{i,j,k}^{(L-1)} \mathbf{m}_{i,j,k}^{(L-1)} + \mathbf{W}_{\tau(i),res,k}^{(L-1)} \mathbf{h}_{i,k}^{(L-1)} \right), \quad (7)$$

where $\mathbf{h}_i^{(L)}$ is the final representations of the heterogeneous graph. We only take out the column nodes representations as the primary representations $\mathbf{h}_c^o$ for later feature propagation.

C. Edge Generation Module

In this part, we will describe how to establish edges $\mathcal{E}_{col-col}$ between column nodes. The edges contain two parts: inside tables and across tables. The detailed procedures can be summarized as the following.

- Edges inside tables. As described in Section IV-A, edges inside tables are fixed and will never be changed once it is created. The adjacent matrix of those edges can be illustrated below:

$$\mathbf{A}_{ij}^{inside} = \begin{cases} 1, & \text{if condition,} \\ 0, & \text{otherwise,} \end{cases} \quad (8)$$

where *condition* means that column i and column j are from the same table.

- Edges across tables. Those edges aim to explore similar column nodes and will be dynamically updated during the training process. Inspired by HGSL [31] producing feature similarity graph, we leverage the primary column representations to construct edges. Without loss of generality, we choose cosine similarity as the similarity measurement [25], [28]. For any two column node i and j , the similarity score between them is:

$$\Gamma(\mathbf{h}_{c_i}, \mathbf{h}_{c_j}) = \cos(\mathbf{h}_{c_i}^o, \mathbf{h}_{c_j}^o), \quad (9)$$

where $\mathbf{h}_{c_i}^o$ and $\mathbf{h}_{c_j}^o$ are the primary column representations after the First Stage of Message Passing. In the real implementation, we sample a batch of nodes as shown in Line 2 of Algorithm 1; thus, node index i is selected from the batch nodes. Node index j is selected from randomly sampled n_s column nodes, in order to save time and memory.

The corresponding adjacent matrix is:

$$\mathbf{A}_{ij}^{across} = \begin{cases} 1, & \text{if } \Gamma(\mathbf{h}_{c_i}, \mathbf{h}_{c_j}) \geq \tau, \\ 0, & \text{otherwise,} \end{cases} \quad (10)$$

where τ is a predefined cosine similarity threshold to control the sparsity of the graph.

- Combine the edges. We take the sum of the two adjacent matrices above:

$$\mathbf{A}_{col} = \mathbf{A}^{inside} + \mathbf{A}^{across}, \quad (11)$$

where $\mathbf{A}_{col}$ is our final adjacent matrix for the Second Stage of Message Passing. Note that $\mathbf{A}^{inside}$ and $\mathbf{A}^{across}$ will not have any overlapping edges.

D. Second Stage of Message Passing Module

After the first stage, we get the primary column representations $\mathbf{h}_c^o$. Here we will show how to update $\mathbf{h}_c^o$ to get the final column representations $\mathbf{h}_c^f$. Specifically, we perform one-layer GAT [18] over the adjacent matrix $\mathbf{A}_{col}$ described above:

$$\mathbf{h}_c^f = GAT(\mathbf{A}_{col}, \mathbf{h}_c^o, \mathbf{W}_{gat}), \quad (12)$$

where $\mathbf{h}_c^o$ and $\mathbf{h}_c^f$ are the primary and final node feature matrix. While $\mathbf{h}_c^f$ is the column representations and is used for solving the CT tasks, we generate column pair representations $\mathbf{h}_p^f$ for the RE task as follows:

$$\mathbf{h}_{pij}^f = \sigma(\mathbf{W}_p(\mathbf{h}_{ci}^f \parallel \mathbf{h}_{cj}^f) + \mathbf{b}_p), \quad (13)$$

where i and j denote the i -th and j -th columns of one specific web table.

E. Prediction Module

After the above procedure, we get the final column and column pair representations $\mathbf{h}_c^f$ and $\mathbf{h}_p^f$. Now we will generate the predictions for the CT and RE tasks.

$$\begin{aligned} \hat{y}_c &= \text{Softmax}(\mathbf{W}_c(\mathbf{h}_c^f) + \mathbf{b}_c) \\ &= \frac{\exp(\mathbf{W}_c(\mathbf{h}_{c,k}^f) + \mathbf{b}_c)}{\sum_{v=0}^{V_c} \exp(\mathbf{W}_c(\mathbf{h}_{c,v}^f) + \mathbf{b}_c)}, \forall k = 1, \dots, |V_c|, \\ \hat{y}_p &= \text{Softmax}(\mathbf{W}_p(\mathbf{h}_p^f) + \mathbf{b}_p) \\ &= \frac{\exp(\mathbf{W}_p(\mathbf{h}_{p,k}^f) + \mathbf{b}_p)}{\sum_{v=0}^{V_p} \exp(\mathbf{W}_p(\mathbf{h}_{p,v}^f) + \mathbf{b}_p)}, \forall k = 1, \dots, |V_p|, \end{aligned} \quad (14)$$

where V_c and V_p is the number of classification types on the CT and RE task, respectively.

For some of the datasets (e.g., WikiTable in the experiments), our task is not assigning each sample with a single label, but a multi-label classification problem. We need to change the output layer of Equation 14 to the following:

$$\begin{aligned} \hat{y}_c &= \text{Sigmoid}(\mathbf{W}_c(\mathbf{h}_c^f) + \mathbf{b}_c) \\ &= 1 / (1 + \exp(-(\mathbf{W}_c(\mathbf{h}_c^f) + \mathbf{b}_c))), \\ \hat{y}_p &= \text{Sigmoid}(\mathbf{W}_p(\mathbf{h}_p^f) + \mathbf{b}_p) \\ &= 1 / (1 + \exp(-(\mathbf{W}_p(\mathbf{h}_p^f) + \mathbf{b}_p))). \end{aligned} \quad (15)$$

F. Loss Function and Optimization

The CT and RE tasks can be multi-class or multi-label classification problems. For both problem settings, we adopt cross-entropy loss as our training and optimization objective:

$$\begin{aligned} \mathcal{L}_{CT} &= - \sum_{v \in V_{CT}} y_c^v \log(\hat{y}_c^v) + (1 - y_c^v) \log(1 - \hat{y}_c^v), \\ \mathcal{L}_{RE} &= - \sum_{v \in V_{RE}} y_r^v \log(\hat{y}_r^v) + (1 - y_r^v) \log(1 - \hat{y}_r^v), \end{aligned} \quad (16)$$

where V_{CT} and V_{RE} are classification types for CT and RE tasks, respectively.

In practice, we often train the CT task and the RE task jointly:

$$\mathcal{L} = \gamma \mathcal{L}_{CT} + (1 - \gamma) \mathcal{L}_{RE}, \quad (17)$$

where γ is the hyperparameter to balance the losses from the CT and RE tasks.

V. EXPERIMENTS

In this section, we evaluate the performance of our proposed Tab-HGNN model against various existing methods solving the CT and RE tasks on web tables. We aim to answer the following questions:

- **RQ1:** What is the performance of the proposed Tab-HGNN model compared with the state-of-the-art methods on the CT and RE tasks?
- **RQ2:** How do different kinds of relationships influence the final prediction results?
- **RQ3:** How does the Column-Column relationship improves the prediction accuracy?
- **RQ4:** What is the influence of hyper-parameters on the performance of Tab-HGNN?

A. Datasets

We evaluate the performance of the proposed Tab-HGNN framework and the baselines on two public datasets: WikiTable and VizNet.

- WikiTable is a public multi-label classification dataset released by TURL [9], selected from around 1.65M tables in WikiTablepedia pages and annotated by Freebase. We take a subset of around 5.5K tables which have annotations on both columns and column pairs.
- VizNet is a public multi-class classification dataset from Sato [12], which is on the WebTables corpus from VizNetNet dataset. The semantic types are from the T2Dv2 Gold standard¹.

The detailed statistics of the two datasets are provided in Table I, where # of samples refers to number of columns for the CT task and number of column pairs for the RE task. Note VizNet is only for the CT task. For the train/valid/test split of WikiTable, we follow the split of original papers which is 52306/1550/1462 tables. For VizNet, we leverage 5-fold cross

¹<http://webdatacommons.org/webtables/goldstandardV2.html>

TABLE I
THE STATISTICS OF THE DATASETS.

	Task objective	# of tables	# of samples for CT	# of samples for RE	# of labels for CT	# of labels for RE
WikiTable	Multi-label	55,318	133,104	66,493	237	121
VizNet	Multi-class	78,733	120,609	-	78	-

validation as the original paper, where 80% of the tables are for training, the remaining 20% tables are for testing, and finally, we average the performance scores such as F_1 scores to get the final performance.

B. Baseline Methods

To illustrate the effectiveness of our proposed model, we compare Tab-HGNN² with the state-of-the-art methods for the CT and RE tasks.

- HNN [11]: This method uses RNN to encode word sequences of each cell mention and uses CNN on row-column structure on web tables to form the representations of all the columns.
- SHERLOCK [10]: This method uses hand-crafted features on global statistics, character distribution, word embeddings and paragraph vectors of each column, and uses some feed-forward neural networks to get the final column representations. It only considers the Column-Cell relationship to solve the CT task.
- SATO [12]: This method uses LDA topic model [13] for each table to get the table topic information, together with features of SHERLOCK to describe each column.
- TURL [9]: This method pre-trains a transformer-based model on about 1.65M tables. Then, it performs fine-tuning to get cell representations. Finally, it performs average pooling to get the column representations.
- TCN [8]: This method considers both intra-table and inter-table context of cells, including same value, same schema position and same page topic, and performs average pooling operation for each column to get the final column representations.

We use the open-source implementations of the comparison methods to obtain the results on WikiTable and VizNet. SHERLOCK, SATO and HNN are originally only for the CT task, we concatenate the representations of two columns and use a linear layer to get the column pair representations to fit it into the RE task. For methods implemented on WikiTable datasets, we adopt multi-task training paradigm on both CT and RE tasks. For models designed for multi-class classification problem, we change the output layer into the sigmoid activation layer to fit it into multi-label classification problem.

C. Experiment Setup

Our Tab-HGNN takes the collection of training tables to construct the graph. We inference on the new testing tables

²The code of our method is available at: <https://github.com/sux1ngyu/Tab-HGNN>.

TABLE II
PERFORMANCE COMPARISON ON THE VIZNET DATASET.

Method	Macro F_1	Weighted F_1
HNN	0.702	0.881
Sherlock	0.692	0.867
Sato	<u>0.756</u>	<u>0.902</u>
TCN	0.705	0.873
Tab-HGNN	0.835 (+7.9%)	0.928 (+2.6%)
Tab-HGNN _{random}	0.772	0.886

by constructing edges from training tables to testing tables. We use the validation set for the selection of hyperparameters. The batch size is 16 which means we sample 16 table nodes for each batch. We use the Adam optimizer with learning rate $1e-4$ and linear decay scheduler of no warm up. The number of heads of Tab-HGNN is set to be 6, and the sampling row number for each table is 10. The number of sampled column nodes for constructing dynamic edges is $n_s = 150$. The neighbor number of each node in graph neural network is 20. The threshold for column representations cosine similarity is $\tau = 0.9$ for the VizNet dataset and $\tau = 0.95$ for the WikiTable dataset. The balancing factor of CT and RE tasks is set to be $\gamma = 0.7$. We initialize the features by 300-dimensional GloVe embedding [15], and the edge features are initialized by a uniform distribution for each type. The experiments were conducted on Intel(R) Xeon(R) Silver 4110 CPU with 128GB Memory and Nvidia GeForce RTX 2080 Ti GPU with 12GB Memory.

D. Evaluation Metrics

Following the previous study [12], we use weighted average F_1 scores and macro average F_1 scores to evaluate the performance for both multi-class and multi-label classification problems. Both VizNet and WikiTable are imbalanced on the semantic types. Weighted average F_1 scores will average the per-type F_1 score weighted by the number of samples for each type, while macro average F_1 is the unweighted average. Thus, weighted F_1 scores reflect the overall performance and macro F_1 scores will pay more attention on types with small numbers of samples.

E. Overall Performance (RQ1)

We present the overall performance for Column Type Annotation and Relation Extraction on both datasets VizNet and WikiTable in Table II and III, respectively. We report all the results for the proposed Tab-HGNN and the baseline methods.

TABLE III
PERFORMANCE COMPARISON ON THE WIKITABLE DATASET.

Method	Column Type	Annotation task	Relation Extraction task	
	Macro F ₁	Weighted F ₁	Macro F ₁	Weighted F ₁
HNN	0.365	0.869	0.821	0.899
Sherlock	0.124	0.644	0.505	0.715
TURL	<u>0.525</u>	<u>0.921</u>	0.836	<u>0.925</u>
TCN	0.409	0.892	<u>0.839</u>	0.920
Tab-HGNN	0.541 (+1.6%)	0.923 (+0.2%)	0.842 (+0.3%)	0.926 (+0.1%)
Tab-HGNN _{random}	0.514	0.904	0.815	0.911

TABLE IV
ABLATION STUDIES ON THE VIZNET DATASET.

Method	Macro F ₁	Weighted F ₁
Tab-HGNN	0.835	0.928
Tab-HGNN _{tab-}	0.835 (-0.0%)	0.928 (-0.0%)
Tab-HGNN _{second-}	0.818 (-1.7%)	0.925 (-0.3%)
Tab-HGNN _{dynamic-}	0.833 (-0.2%)	0.926 (-0.2%)

Tab-HGNN_{random} uses random initialization for all the word embeddings, while Tab-HGNN uses GloVe embeddings.

For the baseline methods, we observe that TURL is a highly-competitive method since it leverages 1.65M tables for the pre-training. Sato performs better than TCN and Sherlock on VizNet dataset, since Sato explicitly leverages LDA model [13] to consider the table topic information. Sherlock’s performance drops greatly when the dataset changes from VizNet to WikiTable, which shows that hand-crafted features does not have good generalization. For both the datasets, we can see weighted F₁ scores are much higher than macro F₁ scores, which means both datasets are imbalanced and the types with small sample sizes have poor performance.

For both of the VizNet dataset and the WikiTable datasets, we can observe Tab-HGNN outperforms all the baselines. The improvement in Macro F₁ is generally higher, since both datasets have data imbalance problem. Tab-HGNN fully explore the different relationships, especially Column-Column relationship in the web tables, so that help the classification for the types with scarce samples.

Furthermore, Tab-HGNN_{random} means that we do not rely on any predefined word embeddings, but only randomly initialize the word embeddings. We can notice that Tab-HGNN_{random} also achieves a good performance and is better than many (not all) baseline methods.

F. Ablation Studies (RQ2)

To show the effectiveness of our proposed Tab-HGNN, we compare it with three variants: Tab-HGNN_{tab-}, Tab-HGNN_{second-} and Tab-HGNN_{dynamic-}. Tab-HGNN_{tab-} removes the topics information and evaluates the influence of Column-Table relationship. Tab-HGNN_{second-} removes the Second-Stage Message Passing Module. Tab-HGNN_{dynamic-} removes the dynamically updated edges between columns across different tables, which is A^2 as shown in Equation 10.

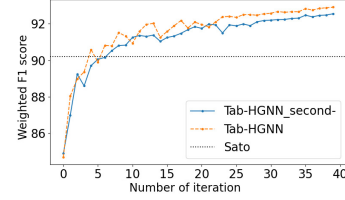


Fig. 3. Training curves of Tab-HGNN and Tab-HGNN_{second-} for CT task on VizNet dataset.

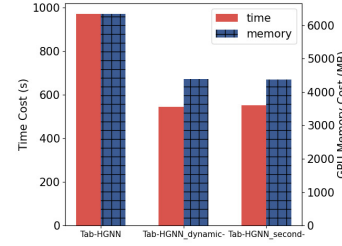


Fig. 4. Time and memory costs for Tab-HGNN and its variants.

The ablation studies are carried on both VizNet and WikiTable datasets as shown in Table IV and V.

From the results in Table IV of the VizNet dataset, we observe that Tab-HGNN_{tab-} performs similarly to Tab-HGNN, since the VizNet dataset itself removes the headers and topics, so that removing table topics will not degenerate the performances. Tab-HGNN_{second-} shows the Column-Column relationship is helpful for the learning of column representations. Tab-HGNN_{dynamic-} shows that both fixed edges and dynamic edges between column nodes are important for the task. Notably, even totally remove the Second-Stage Message Passing Module, Tab-HGNN_{second-} can still have better performances than Sato, which shows the great expressive capability of the proposed Tab-HGNN.

The results of the WikiTable dataset can be seen from Table V. We can observe that different from the VizNet dataset, Tab-HGNN_{tab-} has more performance drops. Tab-HGNN_{second-} and Tab-HGNN_{dynamic-} also have some performance drops compared with Tab-HGNN, which states the importance of the Second-Stage Message Passing Module.

G. Details of Column-Column Relationship (RQ3)

In this part, we will show how the Column-Column relationship improves the final prediction performances. Fig. 3

TABLE V
ABLATION STUDIES ON THE WIKITABLE DATASET.

Method	Column Type Annotation task		Relation Extraction task	
	Macro F_1	Weighted F_1	Macro F_1	Weighted F_1
Tab-HGNN	0.541	0.923	0.842	0.926
Tab-HGNN _{tab-}	0.526 (-1.5%)	0.919 (-0.4%)	0.830 (-1.2%)	0.921 (-0.5%)
Tab-HGNN _{second-}	0.532 (-0.9%)	0.919 (-0.4%)	0.833 (-0.9%)	0.922 (-0.4%)
Tab-HGNN _{dynamic-}	0.538 (-0.3%)	0.921 (-0.2%)	0.840 (-0.2%)	0.925 (-0.1%)

TABLE VI
DETAILS OF EDGE CONNECTIONS BETWEEN COLUMNS ACROSS TABLES.

Iteration	Total Edge Number	Accurate Edge Number	Connection Accuracy (%)
1	209443	67805	32.37
2	31992	30612	95.69
3	26422	25592	96.86
4	23108	22544	97.56
5	20600	20126	97.70
10	16337	16099	98.54
15	13983	13859	99.11
20	14791	14675	99.22
25	14480	14397	99.43
30	13989	13900	99.36
35	14357	14296	99.58
40	13941	13882	99.58

shows the prediction performance as the training process on the testing set of the VizNet dataset. We can observe that when the training begins, there is nearly no differences between Tab-HGNN and Tab-HGNN_{second-}. As the training continues, the differences become larger. It is because at the beginning of the training, most of our constructed edges are not accurate, so that the Second Stage Message Passing Module does not contribute to the performances. As the training continues, our constructed edges are more accurate, making the differences between Tab-HGNN and Tab-HGNN_{second-} become higher. Notably, only after about five iteration, Tab-HGNN can perform better than Sato (the dotted line), showing the ability of our proposed Tab-HGNN model.

Table VI also shows similar results. We record the total edge number, accurate edge number and the connection accuracy for some iterations. At the beginning of the training process, the cosine similarity between column nodes is generally high so we construct many edges, but most edges are wrongly connected, which means we connect two nodes with different labels. As the training continues, most connected edges are accurate, making Tab-HGNN having better performances.

Fig. 4 shows the time and memory costs during the training of Tab-HGNN. We record the cost for each iteration and report the average value. We can observe that establishing dynamic edges takes longer training time than using fixed edges.

H. Parameter Sensitivity (RQ4)

We now examine the effect of hyper-parameters on the final performance of Tab-HGNN. We examine: (1) the maximal row number n_r selected for each table. Since the row number of each table sometimes can be very large, we randomly

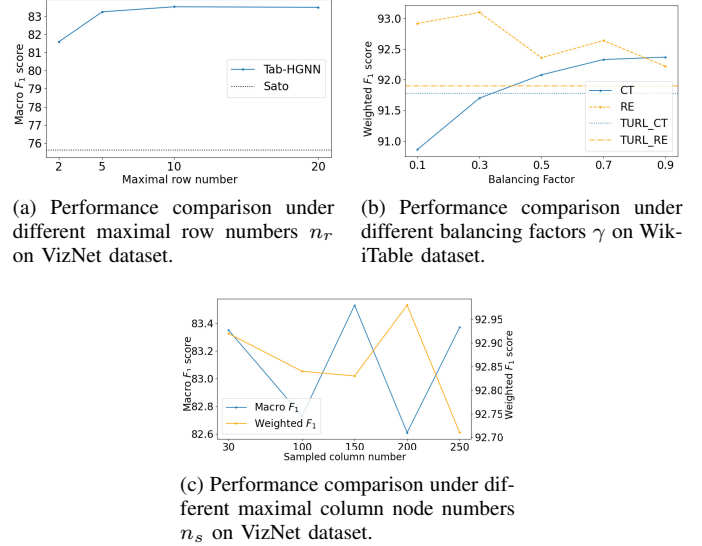


Fig. 5. Parameter sensitivity for maximal row number, balancing factor and maximal column node number.

sample n_r rows for the model training. (2) the balanced factor γ used for balancing the CT and RE losses as shown in Equation 17. (3) the maximal column node number n_s selected for constructing dynamic edges. The results can be seen in Fig. 5a, Fig. 5b and Fig. 5c.

Fig. 5a shows the Macro F_1 scores with different maximal row number n_r selected for each table. The dotted line is the performance of Sato. We can observe that even we only observe two cells for each column, the performance is significantly higher than the baselines. We finally choose $n_r = 10$ since it achieves the best performances.

Fig. 5b shows the Weighted F_1 scores with a different balancing factor γ . We can notice that as γ goes bigger, the model pays more attention on the CT task. Finally, we choose $\gamma = 0.7$ since the average value of Weighted F_1 score of the CT and RE task is the biggest.

Fig. 5c shows the performance for different number of sampled column nodes n_s when constructing dynamic edges. We choose $n_s = 150$ since the average value of Macro F_1 score and Weighted F_1 score is the highest.

VI. CONCLUSION

In this paper, we propose the Tab-HGNN model to deal with the Column Type Annotation and Relation Extraction tasks for web table interpretation. We convert the tables

into a heterogeneous graph, and then utilize Tab-HGNN to learn the column and column pair representations. Extensive experiments show that the proposed Tab-HGNN significantly outperforms the state-of-the-art methods on the CT and RE tasks. In future work, we would like to extend our method to other table interpretation tasks such as Entity Linking and Table Classification, since Tab-HGNN not only learns the column representations, but also learns cell and table representations. Besides, exploring the feasibility of few-shot learning for web table interpretation can be another research direction.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their insightful reviews. This work is supported by the National Key Research and Development Program of China (2022YFE0200500), Shanghai Municipal Science and technology Major Project (2021SHZDZX0102), the Tencent Wechat Rhino-Bird Focused Research Program, and SJTU-Global Strategic Partnership Fund (2021 SJTU-HKUST).

REFERENCES

- [1] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: a collaboratively created graph database for structuring human knowledge," in *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, 2008, pp. 1247-1250.
- [2] J. Lehmann, R. Isele, M. Jakob, A. Jentzsch, D. Kontokostas, P. N. Mendes, and C. Bizer, "Dbpedia—a large-scale, multilingual knowledge base extracted from wikipedia," in *Semantic web*, vol. 6, no. 2, pp. 167-195, 2015.
- [3] X. Ma, and E. Hovy, (2016), "End-to-end sequence labeling via bi-directional lstm-cnns-crf," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.
- [4] Z. Wei, J. Su, Y. Wang, Y. Tian, and Y. Chang, (2019), "A novel cascade binary tagging framework for relational triple extraction," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- [5] D. Zeng, K. Liu, S. Lai, G. Zhou, and J. Zhao, "Relation classification via convolutional deep neural network," in *Proceedings of the 25th international conference on computational linguistics: technical papers*, 2014, pp. 2335-2344.
- [6] M. J. Cafarella, A. Halevy, D. Z. Wang, E. Wu, and Y. Zhang, "Webtables: exploring the power of tables on the web," *Proceedings of the VLDB Endowment*, vol. 1, no. 1, pp. 538-549, Aug. 2008.
- [7] E. Jiménez-Ruiz, O. Hassanzadeh, V. Efthymiou, J. Chen, and K. Srinivas, "SemTab 2019: Resources to Benchmark Tabular Data to Knowledge Graph Matching Systems," *The Semantic Web*, pp. 514-530, 2020.
- [8] D. Wang, P. Shiralkar, C. Lockard, B. Huang, X. L. Dong, and M. Jiang, "TCN: Table Convolutional Network for Web Table Interpretation," *Proceedings of the Web Conference 2021*, Apr. 2021.
- [9] X. Deng, H. Sun, A. Lees, Y. Wu, and C. Yu, "Turl: Table understanding through representation learning," in *Proceedings of the VLDB Endowment*, vol. 14, no. 3, pp. 307-319, Nov. 2020.
- [10] M. Hulsebos et al., "Sherlock: A deep learning approach to semantic data type detection," *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Jul. 2019.
- [11] J. Chen, E. Jimenez-Ruiz, I. Horrocks, and C. Sutton, "Learning Semantic Annotations for Tabular Data," *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, Aug. 2019.
- [12] D. Zhang, M. Hulsebos, Y. Suhara, Ç. Demiralp, J. Li, and W.-C. Tan, "Sato: Contextual semantic type detection in tables," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 1835-1848, Aug. 2020.
- [14] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *ICLR (Workshop Poster)*, 2013.
- [13] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent dirichlet allocation," *Journal of machine Learning research*, 2003, pp. 993-1022.
- [15] J. Pennington, R. Socher, and C. Manning, "Glove: Global Vectors for Word Representation," *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.
- [16] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous Graph Transformer," *Proceedings of The Web Conference 2020*, Apr. 2020.
- [17] H. Iida, D. Thai, V. Manjunatha, and M. Iyyer, "TABBIE: Pretrained Representations of Tabular Data," *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021.
- [18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," *Proceedings of 2018 International Conference on Learning Representations*, 2018.
- [19] Q. Lv et al., "Are we really making much progress? Revisiting, benchmarking and refining heterogeneous graph neural networks," *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, Aug. 2021.
- [20] Z. Wang et al., "TUTA: tree-based transformers for generally structured table pre-training," *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, Aug. 2021.
- [21] L. Zhang, S. Zhang, and K. Balog, "Table2vec: Neural word and entity embeddings for table population and retrieval," *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Jul. 2019.
- [22] P. Yin, G. Neubig, W. Yih, and S. Riedel, "TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data," *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- [23] H. Gao, and S. Ji, "Graph u-nets," in *international conference on machine learning*, pp. 2083-2092, May. 2019.
- [24] K. Sun, Z. Lin, and Z. Zhu, "Multi-Stage Self-Supervised Learning for Graph Convolutional Networks on Graphs with Few Labeled Nodes," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, pp. 5892-5899, Apr. 2020.
- [25] Y. Chen, L. Wu, and M. Zaki, "Iterative deep graph learning for graph neural networks: Better and robust node embeddings," *Advances in neural information processing systems*, 33, 19314-19326, 2020.
- [26] W. Liu, J. He, and S. F. Chang, "Large graph construction for scalable semi-supervised learning," in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, 2010.
- [27] L. Wu, et al, "Scalable global alignment graph kernel using random features: From node embedding to graph embedding," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 1418-1428.
- [28] X. Zhang, and M. Zitnik, "GNNGUARD: defending graph neural networks against adversarial attacks," in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS'20)*, 2020.
- [29] C. Ran, W. Shen, J. Wang, and X. Zhu, "Domain-Specific Knowledge Base Enrichment Using Wikipedia Tables," *2015 IEEE International Conference on Data Mining*, Nov. 2015.
- [30] V. Efthymiou, O. Hassanzadeh, M. Rodriguez-Muro, and V. Christophides, "Matching web tables with knowledge base entities: from entity lookups to entity embeddings," in *International Semantic Web Conference*, 2017, pp. 260-277.
- [31] J. Zhao, X. Wang, C. Shi, B. Hu, G. Song, and Y. Ye, "Heterogeneous graph structure learning for graph neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35, No. 5, pp. 4697-4705, 2021.
- [32] T. N. Kipf, and M. Welling, "Semi-supervised classification with graph convolutional networks," *Proceedings of the 5th International Conference on Learning Representations*, 2016.
- [33] Q. Wang, Y. Fang, A. Ravula, F. Feng, X. Quan, and D. Liu, "WebFormer: The Web-page Transformer for Structure Information Extraction," in *Proceedings of the ACM Web Conference*, 2022, pp. 3124-3133.
- [34] Z. Zhang, "Effective and efficient semantic table interpretation using tableminer+," *Semantic Web*, Vol. 8, No. 6, pp. 921-957, 2017.
- [35] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, and P. S. Yu, "Heterogeneous graph attention network," in *The world wide web conference*, 2019, pp. 2022-2032.